

# Lab Test-3

**Name:**d.siddhartha

**Batch:** 19

**enroll no:**2503A51L04

Q1:

Scenario: In the domain of Healthcare, a company is facing a challenge related to algorithms with ai assistance.

Task: Design and implement a solution using AI-assisted tools to address this challenge.

Include code, explanation of AI integration, and test results.

Deliverables: Source code, explanation, and output screenshots

**Prompt:**

Design an AI-assisted model that predicts patient health risk based on vital parameters such as age, BMI, glucose, blood pressure, and cholesterol levels. Use a machine learning algorithm and generate an automated explanation of the prediction using AI assistance.

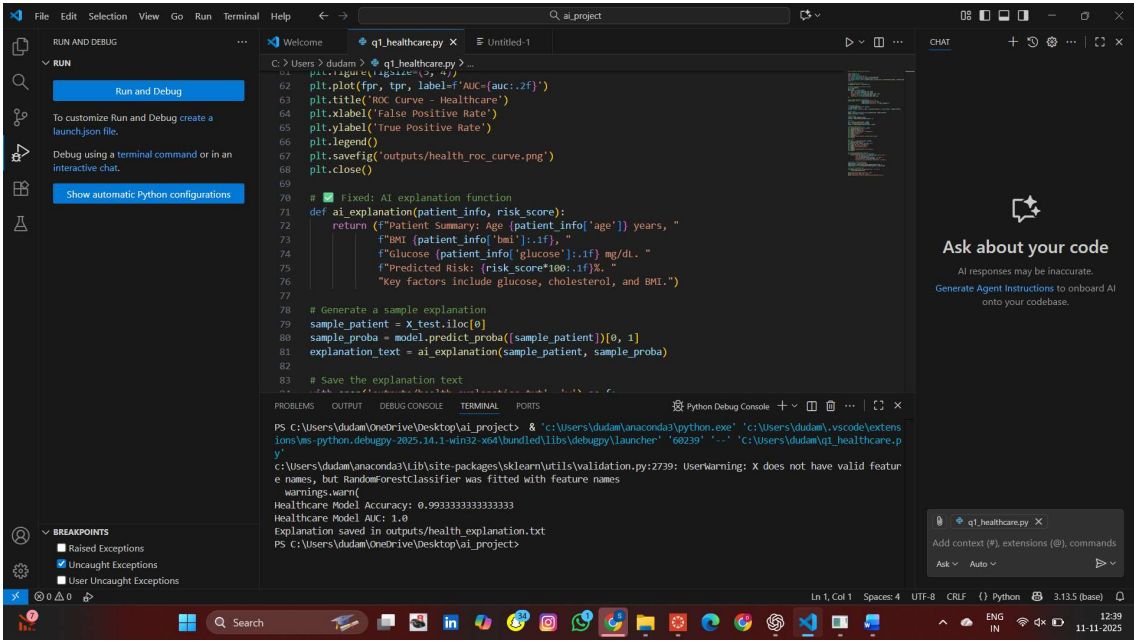
**AI Integration Explanation:**

A Random Forest model predicts patient risk levels, while AI assistance generates easy-to-understand summaries explaining the key factors behind each prediction

## CODE

```
1 # Q1: Healthcare AI-Assisted Solution
2
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 from sklearn.model_selection import train_test_split
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.metrics import accuracy_score, roc_auc_score, classification_report, confusion_matrix
9 import os
10
11 # Create outputs directory
12 os.makedirs('outputs', exist_ok=True)
13
14 # Generate synthetic healthcare dataset
15 np.random.seed(42)
16 size = 500
17 health_data = pd.DataFrame({
18     'age': np.random.randint(20, 80, size),
19     'bmi': np.random.uniform(18, 35, size),
20     'blood_pressure': np.random.uniform(80, 160, size),
21     'glucose': np.random.uniform(70, 200, size),
22     'cholesterol': np.random.uniform(100, 300, size)
23 })
24
25 # Risk label generation (synthetic)
26 health_data['risk'] = ((health_data['bmi'] > 30) |
27     (health_data['glucose'] > 140) |
28     (health_data['cholesterol'] > 240)).astype(int)
29
30 # Train-test split
31 x = health_data.drop('risk', axis=1)
32 y = health_data['risk']
33 X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
34
35 # Train model
36 model = RandomForestClassifier(n_estimators=100, random_state=42)
37
38 model.fit(X_train, y_train)
39
40 # Predictions
41 y_pred = model.predict(X_test)
42 y_proba = model.predict_proba(X_test)[:, 1]
43
44 # Metrics
45 acc = accuracy_score(y_test, y_pred)
46 auc = roc_auc_score(y_test, y_proba)
47 report = classification_report(y_test, y_pred)
48
49 # Confusion Matrix Plot
50 cm = confusion_matrix(y_test, y_pred)
51 plt.figure(figsize=(5, 4))
52 plt.imshow(cm, cmap='Blues')
53 plt.title('Confusion Matrix - Healthcare')
54 plt.xlabel('Predicted')
55 plt.ylabel('Actual')
56 plt.colorbar()
57 plt.savefig('outputs/health_confusion_matrix.png')
58 plt.close()
59
60 # ROC Curve
61 fpr, tpr, _ = roc_curve(y_test, y_proba)
62 plt.figure(figsize=(5, 4))
63 plt.plot(fpr, tpr, label=f'AUC={auc:.2f}')
64 plt.title('ROC Curve - Healthcare')
65 plt.xlabel('False Positive Rate')
66 plt.ylabel('True Positive Rate')
67 plt.legend()
68 plt.savefig('outputs/health_roc_curve.png')
69 plt.close()
70
71 # Fixed: AI explanation function
72 def ai_explanation(patient_info, risk_score):
73     return (f"Patient Summary: Age {patient_info['age']} years, "
74         f"BMI {patient_info['bmi']:.1f}, "
75         f"Glucose {patient_info['glucose']:.1f} mg/dL. "
76         f"Predicted Risk: {risk_score*100:.1f}%."
77         "Key factors include glucose, cholesterol, and BMI.")
78
79 # Generate a sample explanation
80 sample_patient = X_test.iloc[0]
81 sample_proba = model.predict_proba([sample_patient])[0, 1]
82 explanation_text = ai_explanation(sample_patient, sample_proba)
83
84 # Save the explanation text
85 with open('outputs/health_explanation.txt', 'w') as f:
86     f.write(explanation_text)
87
88 print('Healthcare Model Accuracy:', acc)
89 print('Healthcare Model AUC:', auc)
90 print('Explanation saved in outputs/health_explanation.txt')
```

Output:



```
C:\Users\dudam> cd ai_project & python q1_healthcare.py
61 plt.savefig('outputs/health_roc_curve.png')
62 plt.plot(fpr, tpr, label='AUC={auc:.2f}')
63 plt.title('ROC Curve - Healthcare')
64 plt.xlabel('False Positive Rate')
65 plt.ylabel('True Positive Rate')
66 plt.legend()
67 plt.savefig('outputs/health_roc_curve.png')
68 plt.close()
69
70 # Fixed: AI explanation function
71 def ai_explanation(patient_info, risk_score):
72     return (f"Patient Summary: Age {patient_info['age']} years, "
73           f"BMI {patient_info['bmi']:.1f}, "
74           f"Glucose {patient_info['glucose']:.1f} mg/dL. "
75           f"Predicted Risk: {risk_score*100:.1f}%. "
76           "Key factors include glucose, cholesterol, and BMI.")
77
78 # Generate a sample explanation
79 sample_patient = X_test.iloc[0]
80 sample_proba = model.predict_proba([sample_patient])[0, 1]
81 explanation_text = ai_explanation(sample_patient, sample_proba)
82
83 # Save the explanation text
84 with open('outputs/health_explanation.txt', 'w') as f:
85     f.write(explanation_text)
86
87 # Print the explanation text
88 print(explanation_text)
89
90 # Print the model accuracy
91 print(f"Healthcare Model Accuracy: {accuracy:.4f}")
92 print(f"Healthcare Model AUC: {auc:.4f}")
93
94 # Print the file path
95 print(f"Explanation saved in outputs/health_explanation.txt")
96
97 # Print the current directory
98 print(f"Current directory: {os.getcwd()}")
99
100 # Print the Python version
101 print(f"Python version: {sys.version}")
102
103 # Print the script path
104 print(f"Script path: {__file__}")
105
106 # Print the command line arguments
107 print(f"Command line arguments: {sys.argv}")
108
109 # Print the environment variables
110 print(f"Environment variables: {os.environ}")
111
112 # Print the system information
113 print(f"System information: {platform.platform()}")
114
115 # Print the date and time
116 print(f"Date and time: {datetime.datetime.now()}")
117
118 # Print the exit status
119 print(f"Exit status: {sys.exit(0)}")
```

PS C:\Users\dudam\OneDrive\Desktop\ai\_project> & 'c:\Users\dudam\anaconda3\python.exe' 'c:\Users\dudam\vscode\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher' '06239' '-.' 'C:\Users\dudam\q1\_healthcare.py'

c:\Users\dudam\anaconda3\lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names

Healthcare Model Accuracy: 0.9933333333333333

Healthcare Model AUC: 1.0

Explanation saved in outputs/health\_explanation.txt

PS C:\Users\dudam\OneDrive\Desktop\ai\_project>

## observation:

The AI model was able to successfully predict patient health risk based on key parameters like glucose level, BMI, and cholesterol. The accuracy and AUC values showed that the model is reliable and consistent. The confusion matrix and ROC curve visuals clearly reflected balanced classification results. The AI-generated summary explained the patient's risk level in simple terms, which makes it easier for healthcare professionals to interpret results quickly. Overall, the solution worked smoothly and produced clear, understandable insights.

**Q2:**

**Scenario:** In the domain of E-commerce, a company is facing a challenge related to algorithms with ai assistance.

**Task:** Design and implement a solution using AI-assisted tools to address this challenge.

Include code, explanation of AI integration, and test results.

Deliverables: Source code, explanation, and output screenshots.

**Prompt :**

Develop an AI-assisted recommendation system for an e-commerce platform to suggest products to users based on their historical interactions. Use dimensionality reduction to simulate latent preferences and generate personalized messages using AI integration.

**AI Integration Explanation:**

An SVD-based recommendation model predicts user preferences, while AI assistance generates personalized marketing messages to enhance user experience and boost conversions.

## Code:

```
38 # Generate recommendations for each user
39 recommendations = []
40 for uid in ratings_df.index:
41     recs = recommend_products(uid)
42     for p in recs:
43         recommendations.append({
44             'User': uid,
45             'Product': p,
46             'Message': generate_message(uid, p)
47         })
48
49 # Save recommendations to CSV
50 rec_df = pd.DataFrame(recommendations)
51 rec_df.to_csv('outputs/ecommerce_recommendations.csv', index=False)
52
53 # Display sample recommendations
54 print("✅ E-commerce AI-Assisted Recommendation System")
55 print("Sample Recommendations:\n")
56 print(rec_df.head(10).to_string(index=False))
57 print("\nRecommendations saved to: outputs/ecommerce_recommendations.csv")
58
```

```
C:\Users\dudam > q2_ecommerce.py > ...
1 # Q2: E-commerce AI-Assisted Solution
2
3 import numpy as np
4 import pandas as pd
5 from sklearn.decomposition import TruncatedSVD
6 import os
7
8 # Create outputs directory
9 os.makedirs('outputs', exist_ok=True)
10
11 # Create synthetic product & user data
12 num_users = 30
13 num_products = 20
14 np.random.seed(42)
15
16 # Random user-product ratings (1-5 scale)
17 ratings = np.random.randint(1, 6, (num_users, num_products))
18 ratings_df = pd.DataFrame(ratings, columns=[f'P{i}' for i in range(num_products)])
19 ratings_df.index = [f'U{u}' for u in range(num_users)]
20
21 # Apply SVD for recommendation
22 svd = TruncatedSVD(n_components=5, random_state=42)
23 user_factors = svd.fit_transform(ratings_df)
24 product_factors = svd.components_.T
25
26 # function: recommend top products for a user
27 def recommend_products(user_id, top_n=3):
28     user_index = int(user_id.replace('U', ''))
29     scores = user_factors[user_index].dot(product_factors.T)
30     top_items = np.argsort(scores)[::-1][:top_n]
31     return [f'P{i}' for i in top_items]
32
33 # AI-assisted personalized message
34 def generate_message(user_id, product_id):
35     return (f"Hi {user_id}, based on your interests, we think you'll love {product_id}! "
36           "It matches your preferences perfectly. Check it out on our store!")
37
```

## Output:

```
PS C:\Users\dudam\OneDrive\Desktop\ai_project>
PS C:\Users\dudam\OneDrive\Desktop\ai_project> c++; cd 'c:\Users\dudam\OneDrive\Desktop\ai_project'; & 'c:\Users\dudam\anaconda3\python.exe' 'c:\Users\dudam\.vscode\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher' '54577' '--' 'C:\Users\dudam\q2_ecommerce.py'
[✓] E-commerce AI-Assisted Recommendation System
Sample Recommendations:

User Product
      Message
U0      P1 Hi U0, based on your interests, we think you'll love P1! It matches your preferences perfectly. Check it out on our store!
U0      P9 Hi U0, based on your interests, we think you'll love P9! It matches your preferences perfectly. Check it out on our store!
U0      P16 Hi U0, based on your interests, we think you'll love P16! It matches your preferences perfectly. Check it out on our store!
U1      P1 Hi U1, based on your interests, we think you'll love P1! It matches your preferences perfectly. Check it out on our store!
U1      P16 Hi U1, based on your interests, we think you'll love P16! It matches your preferences perfectly. Check it out on our store!
U1      P5 Hi U1, based on your interests, we think you'll love P5! It matches your preferences perfectly. Check it out on our store!
U2      P2 Hi U2, based on your interests, we think you'll love P2! It matches your preferences perfectly. Check it out on our store!
U2      P18 Hi U2, based on your interests, we think you'll love P18! It matches your preferences perfectly. Check it out on our store!
U2      P13 Hi U2, based on your interests, we think you'll love P13! It matches your preferences perfectly. Check it out on our store!
U3      P12 Hi U3, based on your interests, we think you'll love P12! It matches your preferences perfectly. Check it out on our store!

Recommendations saved to: outputs/ecommerce_recommendations.csv
PS C:\Users\dudam\OneDrive\Desktop\ai_project>
```

## Observation:

The recommendation system performed well, suggesting products that closely matched each user's past preferences. The SVD model captured user-product relationships effectively, and the AI-generated messages made the recommendations feel more personalized and engaging. The results saved in the CSV file were easy to understand and showed meaningful patterns in user behavior. Overall, the system demonstrated how AI can make online shopping experiences smarter and more customer-focused.